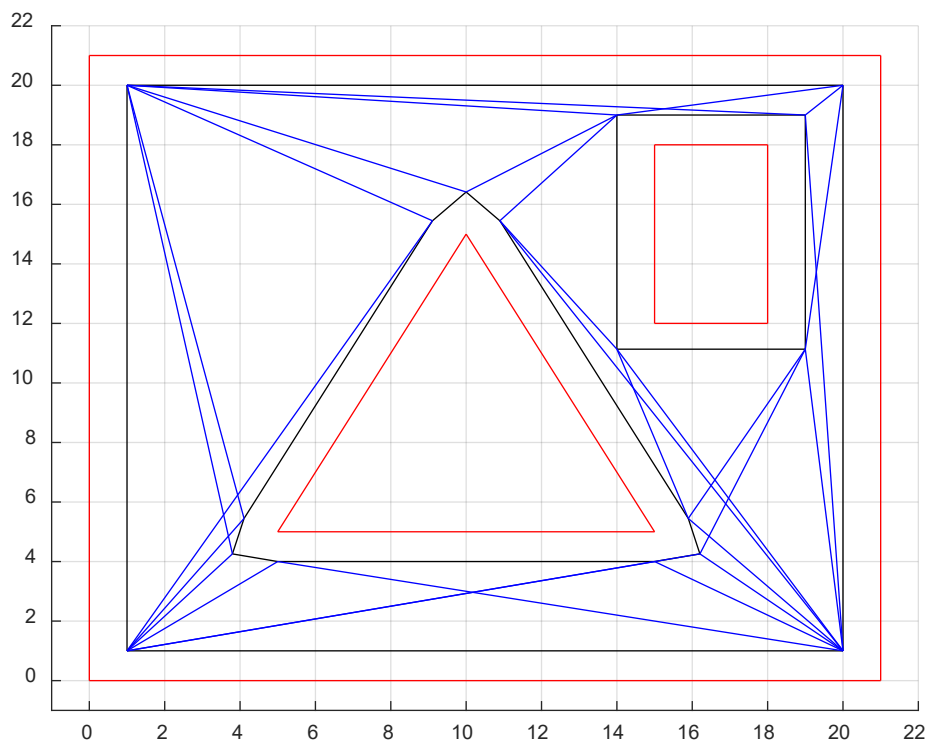


PRÁCTICA DE ROBÓTICA

PLANIFICACIÓN DE MOVIMIENTO



Ranko Zotovic Stanisic

1. Introducción

El objetivo de esta práctica es el repaso y la visualización de algunos conceptos de planificación de movimiento vistos en clase. Concretamente, el modelado del entorno su expansión, el diagrama de visibilidad, el algoritmo de Dijkstra y los métodos de PRM (probabilistic roadmap) y RRT (Rapidly exploring random trees).

Como herramienta de simulación se utilizará el Matlab.

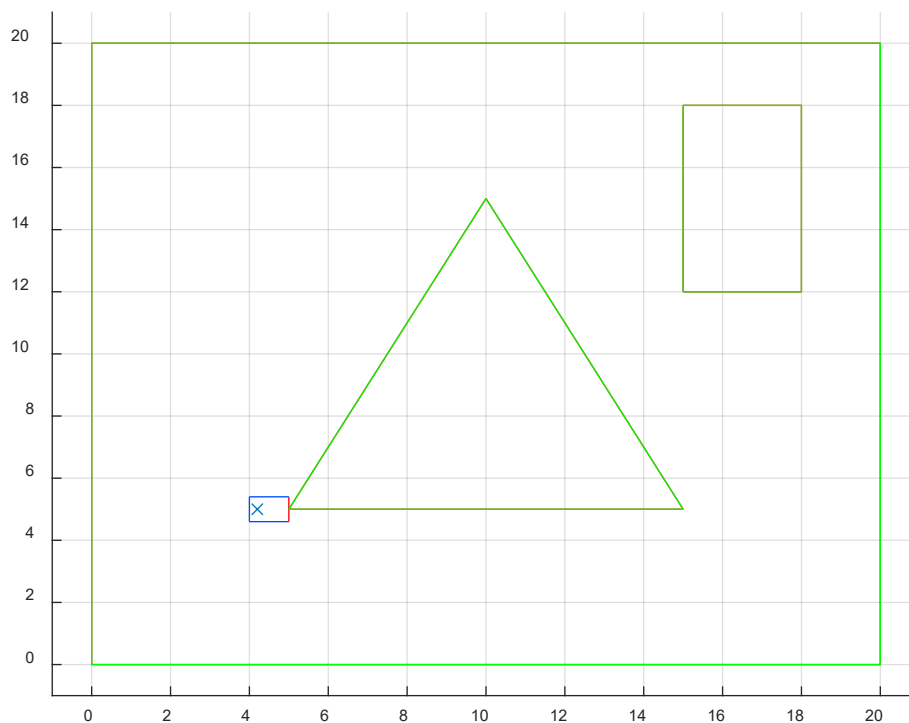
En un entorno sencillo, los alumnos podrán probar estos conceptos y familiarizarse con ellos.

2. Modelado y expansión del entorno

La mayoría de los algoritmos de planificación de camino están implementados para un robot puntual, sin dimensiones. Las dimensiones del entorno complican enormemente la planificación.

Por estas razones, se procede a la expansión de entorno, que consiste básicamente en el aumento del entorno original de la dimensión del robot, de forma a que no pueda impactar con los obstáculos.

En la figura de abajo el robot está representado en azul y rojo, y los obstáculos en verde. La cruz en el robot representa la localización del sistema de coordenadas de este.



Se puede apreciar que el robot entra en contacto con el entorno a pesar de que su sistema de coordenadas esté a cierta distancia. Para tratar este problema, es necesario expandir el entorno de las dimensiones del robot.

Primero, para representar gráficamente el entorno introducir:

```
entorno_practica % Declarar la matriz del entorno.
```

```
figure
```

```
dibuent(ent,[0,1,0]) % Representar el entorno
```

```
axis([-1 22 -1 22]) % Definir los límites.
```

```
hold on
```

```
grid
```

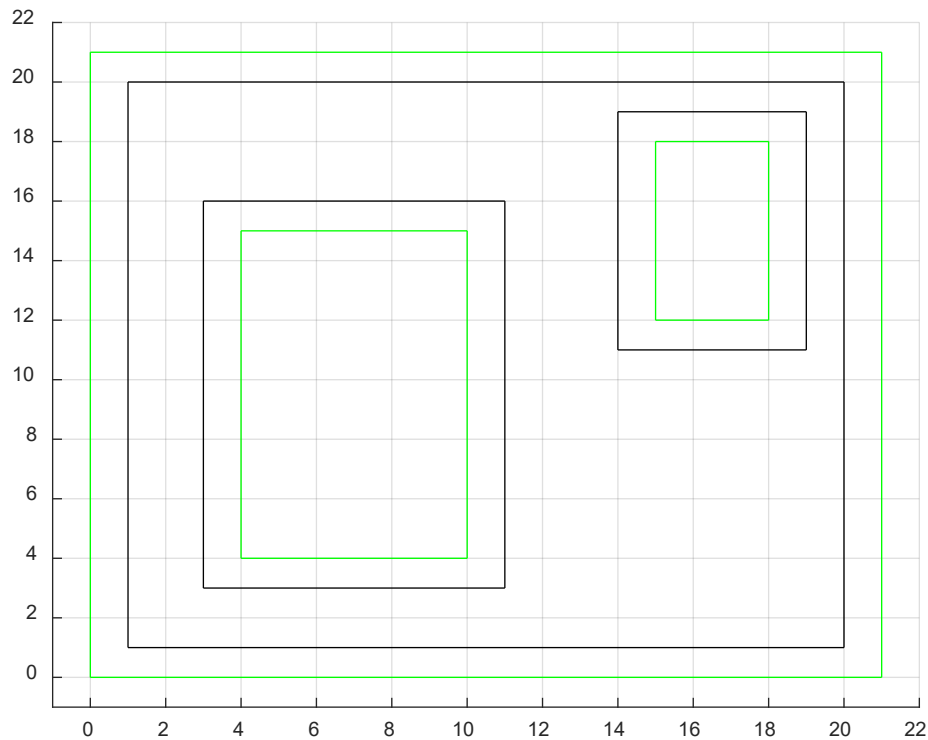
Una vez representado el entorno, se va a expandir el entorno de las dimensiones del robot, en este caso un metro. Esto se corresponde a la distancia máxima entre cualquier punto del robot Robuter II y su origen de coordenadas, Para ello, teclear:

```
expandido=expandir_entorno(ent,1) % Generación del entorno expandido de un metro.
```

Para representarlo gráficamente, por ejemplo, en negro, introducir:

```
dibuent(expandido,[0 0 0])
```

Se obtiene la siguiente figura



En verde está representado el entorno original, y en negro el entorno expandido. Si la planificación se hace en el expandido, el robot se podrá tratar como un punto sin dimensiones y no habrá peligro de colisiones.

Se pueden apreciar algunas cosas:

- El segundo obstáculo se ha unido con las paredes de la habitación. Se podría hacer un paso más y fusionarlos en un solo objeto. Sin embargo, esto no ha sido implementado en la versión actual dado que no es el objetivo primordial de la práctica.
- El objeto obtenido al expandir el obstáculo en forma de triángulo tiene una forma no convencional. En realidad, tiene nueve ángulos. Esta forma se ha obtenido mediante el método de expansión utilizado. Existen otros que optimizan más el espacio, pero son más complejos.
- Una vez obstáculo, la superficie dentro de los obstáculos se considerará espacio de obstáculos (Cobst) y el resto espacio libre (Cfree). De esta forma, el robot se podrá considerar como un punto sin dimensiones para la planificación de movimiento. Cabe resaltar que, en la versión actual del software, la función de expansión del entorno no funciona con objetos cóncavos ni con objetos entre los cuales la distancia es menor que la distancia a la que se expanden.

3. Diagrama de visibilidad

El siguiente paso en la planificación de movimiento puede ser la implementación del diagrama de visibilidad.

Consiste en unir todos los vértices (tanto de la habitación como de los obstáculos) entre ellos con líneas, pero descartando aquellas líneas que corten algún obstáculo. De esta forma se pueden determinar los caminos libres.

Para generar las rectas introducir:

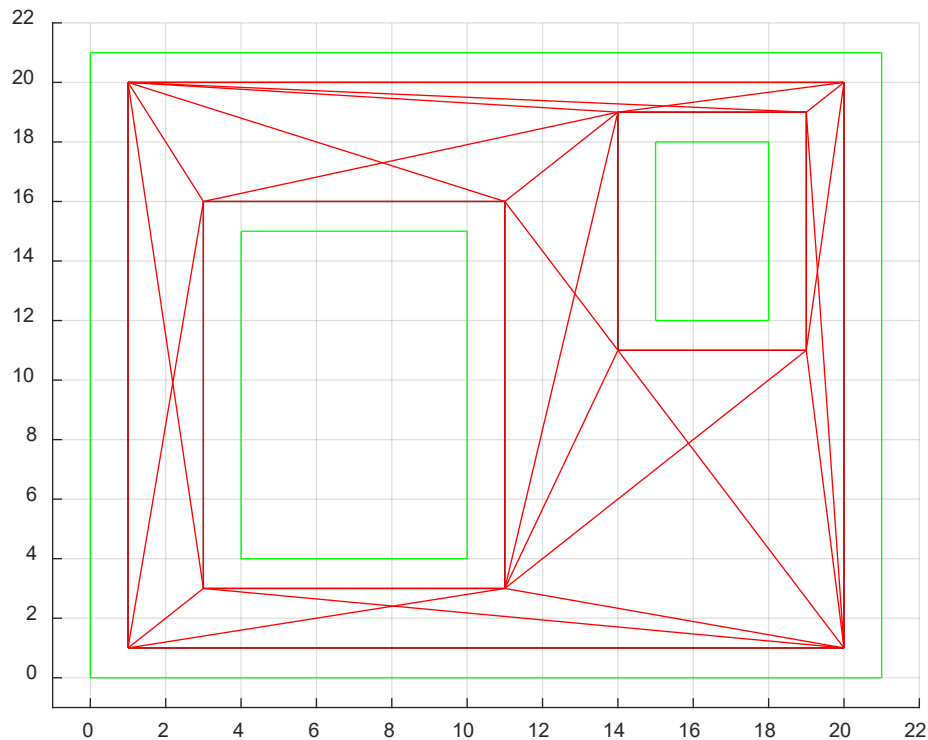
```
[rectas, matriz]=rectas_visibilidad_expandido(expandido)
```

Esta función también devuelve la matriz de costes de los arcos que unen los nodos. En este caso se corresponden a las distancias entre los nodos. Esta matriz se puede utilizar para la búsqueda del camino óptimo.

Para representar gráficamente las rectas, por ejemplo, en rojo, teclear:

```
lineas=dib_rectas(rectas,[1 0 0])
```

Se obtiene la siguiente figura:



En rojo está representadas las líneas de visibilidad.

El siguiente paso podría ser la búsqueda del camino más corto. Para ello se ha implementado el algoritmo de Dijkstra.

Para encontrar el camino más corto, por ejemplo, entre el primer nodo y el octavo, introducir:

```
[Camino, Coste]=Dijkstra(matriz,1,8)
```

Esta función devuelve el camino más corto (secuencia de nodos por los que se debe pasar) así como su coste.

Para representar el camino gráficamente, primero se deben extraer los vértices introduciendo:

```
[vertices]=extrae_vertices(expandido)
```

```
Trayecto = dibujar_camino(Camino,vertices,[0 0 1])
```

El siguiente paso podría ser la introducción del punto de partida del robot, así como del objetivo.

Para ejecutar la nueva implementación del diagrama de visibilidad debemos volver a crear el entorno original y expandido en una nueva figura y ejecutar el siguiente código:

```
ini= [2,2];
fin= [13,14]; %% Se introducen el punto inicial y el final.
[rectas, matriz, vertices]=rectas_visibilidad_expandido_manual(expandido, ini,fin);
lineas=dib_rectas(rectas,[1 0 0]);
```

```
[Camino, Coste]=Dijkstra_manual(matriz);
Trayecto = dibujar_camino_manual(Camino,vertices,[0 0 1]);
```

```

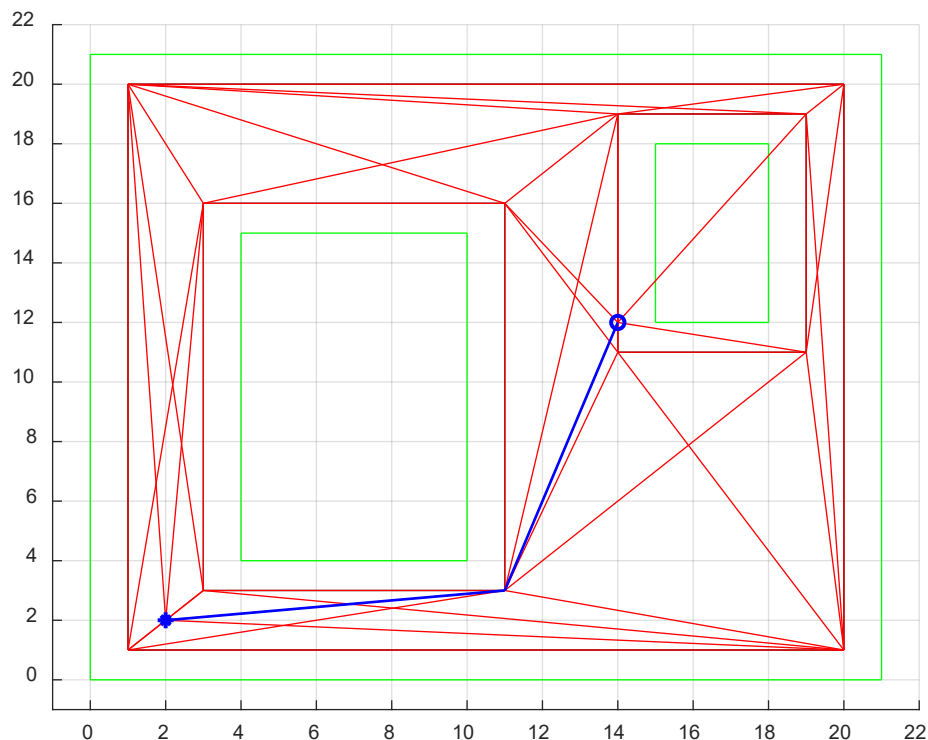
%-----
% DIAGRAMA DE VISIBILIDAD ASIGNANDO NODO INICIAL Y FINAL
%-----
% [rectas, matriz]=rectas_visibilidad_expandido(expandido);
% lineas=dib_rectas(rectas,[1 0 0]);
%
% [Camino, Coste]=Dijkstra(matriz,1,8);
% [vertices]=extrae_vertices(expandido);
% Trayecto = dibujar_camino(Camino,vertices,[0 0 1]);

```

En esta nueva versión del diagrama de visibilidad se permite introducir como parámetro (ini y fin) los puntos origen y destino. Al introducir estos puntos se les trata como unos nuevos vértices para poder realizar el diagrama. Para realizar esto se ha modificado la función de “rectas_visibilidad_expandido” pasando a ser “rectas_visibilidad_expandido_manual” para poder usar como parámetros los puntos.

También se ha modificado el algoritmo de Dijkstra pasando a ser “Dijkstra_manual” en el cual ya no se le pasan los nodos origen y destino. Por último, se ha modificado la función de “dibujar_camino” pasando a ser “dibujar_camino_manual” para dibujar correctamente el camino al elegir manualmente los puntos.

El resultado es el siguiente:



- **Anotar el coste del camino para compararlo con el obtenido con otros métodos.**

Cabe resaltar que este método funciona muy bien con entornos poligonales 2D, pero peor con otros.

4. Método de mapa probabilístico (Probabilistic roadmap-PRM)

Este método de búsqueda del camino óptimo consiste en dos fases: la de aprendizaje y la de consulta. La primera consiste en la formación del mapa y, si es necesario, su expansión. Se procede a la expansión si se demuestra que el número de nodos inicial no es suficiente.

La fase de aprendizaje consta de los siguientes pasos:

1. Generar de forma aleatoria un cierto número de puntos.
2. Descartar aquellos que estén dentro de un obstáculo (Cobst).
3. Unir los restantes por arcos.
4. Descartar los arcos que crucen algún obstáculo.
5. Formar una matriz de costes con los arcos restantes.

En la fase de consulta se realiza lo siguiente:

1. Se añaden los nodos de inicio y objetivo al grafo.
2. Se conectan con los demás nodos, descartando aquellas conexiones que crucen algún obstáculo.
3. Se pone al día la matriz de costes con los nuevos nodos-
4. Se encuentra el camino de menos coste (Dijkstra, A*, etc).
5. Si el algoritmo falla al encontrar la solución, se vuelve a la fase de aprendizaje, generando más puntos si es necesario.

Par el desarrollo de la práctica, se puede empezar por, si es necesario, introduciendo el entorno:

```
entorno_practica    % Declarar la matriz del entorno.
```

```
figure
```

```
dibuent(ent,[0,1,0]) % Representar el entorno
```

```
axis([-1 22 -1 22]) % Definir los limites.
```

```
hold on
```

```
grid
```

```
expandido=expandir_entorno(ent,1) % Generación del entorno expandido de un metro.
```

```
dibuent(expandido,[0 0 0])
```

El siguiente paso podría ser la generación de un número determinado de puntos aleatorios. Se va a empezar por diez puntos. Cabe resaltar que es necesario delimitar la zona en la que se generan los puntos. En este caso, se trabaja en un área de 21 x21.

Se puede ejecutar la función *Generar_puntos_aleatorios(Npuntos, xmin, xmax,ymin,ymax)*. Npuntos es el número de puntos que se quiere generar, mientras xmin, max, ymin, e ymax delimitan el entorno en el que queremos que se generen.

Se puede introducir:

```
[Puntos] = Generar_puntos_aleatorios(10,0,21,0, 21)
```

Para representarlos, por ejemplo en forma de cruz, se ejecuta:

```
plot(Puntos(:,1),Puntos(:,2),'x')
```

Dado que los puntos son generados de forma aleatoria, es probable que algunos de ellos estén dentro de un obstáculo. Para diferenciar los puntos alcanzables de los que están dentro de obstáculos, introducir:

```
[Ptos_free,Ptos_Obst] = Delimita_Cfree(Puntos, expandido)
```

El siguiente paso es la generación del diagrama de visibilidad. Se trata de encontrar todas las líneas rectas que unen los puntos sin cruzar ningún obstáculo. Esto se puede lograr introduciendo:

```
[rectas matriz_costes]=visibilidad_puntos(expandido,Ptos_free)
```

Para representar gráficamente estas líneas teclear:

```
lineas=dib_rectas2(rectas,color)
```

Cabe resaltar que la función *visibilidad_puntos* devuelve también la matriz de costes que se utilizará posteriormente para planificar el camino.

Dado que este método es probabilístico, se puede dar el caso de que haya nodos no conectados, aunque exista la posibilidad. En este caso se debe repetir el proceso, si es necesario aumentando el número de puntos generados.

Probar generar cinco veces cinco puntos aleatorios, cinco veces diez, cinco veces 20 y cinco veces 50. Representar gráficamente el diagrama de visibilidad en cada caso.

A continuación, se va a proceder a planificar la trayectoria óptima. Para empezar se van a definir el punto de partida (start) y el objetivo (goal). Obviamente, ambos tienen que pertenecer al espacio libre.

Se va a elegir, por ejemplo, el start en el punto (2,2) y el goal en (13,14).

Introducir:

```
start =[2,2];  
goal=[13,14];
```

Para representarlos gráficamente, por ejemplo, en forma de círculo y cuadrado, introducir:

```
plot(start(1),start(2),'o')  
plot(goal(1), goal(2),'s')
```

La forma más fácil de buscar el camino más corto del inicio al objetivo es, para empezar, añadiéndolos a la lista de puntos:

```
Puntos=[start;goal;Puntos]
```

Cabe resaltar que, para este ejemplo, se ha optado por poner al start de primero y el goal de segundo. Cualquier otra elección puede ser válida.

A continuación, se pueden volver a generar las rectas con el número de puntos aumentado:

```
[rectas,matriz_costes]=visibilidad_puntos(entorno, Ptos_free)
lineas=dib_rectas2(rectas,[0.5 0.5 0])
```

Por último, para buscar el camino óptimo, simplemente se ejecuta el algoritmo de Dijkstra:

```
[Camino, Coste]=Dijkstra(matriz_costes,1,2)
```

Recordar los nodos 1 y 2 son el start y el goal respectivamente.

Para representar gráficamente el camino introducir:

```
Trayecto = dibujar_camino(Camino, Ptos_free,[1 0 0]).
```

Cabe resaltar que, dado que el método PRM es probabilístico, cada vez que se repite puede dar un resultado diferente. Puede que el camino encontrado no sea el más corto posible. Hasta puede ser que no se encuentre el camino, aunque este exista. Sin embargo, si se repite suficientes veces con suficientes puntos generados, terminará encontrado el camino si este existe. En cuanto a la optimización del camino, a más puntos se generan es mayor la probabilidad de encontrar el camino más corto. Por otra parte, a más puntos, mayor coste computacional.

Hacer tres experimentos generado 10, 20 y 50 nodos (tres para cada caso). Representar gráficamente los puntos, las rectas y el camino óptimo. Compara los costes y el tiempo de cálculo en cada caso utilizando las funciones de Matlab *tíc* y *toc*.

5. Método RRT (Rapidly exploring random trees- Árboles de exploración rápida aleatoria)

Este algoritmo consiste en generar un árbol de puntos conectados entre ellos. Se parte del punto inicial del robot y se van generando nodos aleatorios. Para cada punto nuevo creado aleatoriamente se busca el nodo más cercano del árbol. Se traza la línea que los une. Se calcula el punto de esta línea que está a una distancia predefinida (en esta práctica un metro) del nodo más cercano del árbol. Si este punto no intersecta ningún obstáculo, se añade al árbol.

De esta forma, el árbol se expande hasta alcanzar el objetivo.

5.1. RRT básico

Cabe resaltar que este método tiene varias versiones. Este apartado está dedicado a la más básica.

Se trata de un método es aleatorio, por prueba y error. Posiblemente no sea ni el más rápido ni el óptimo, pero siempre encuentra el camino si este existe.

Cabe resaltar que en esta práctica este método se implementado mediante la programación orientada a objetos.

Para empezar, se deben inicializar las variables. Ejecutar el fichero *inicializar_RRT.m* . Leer el fichero y entenderlo.

Toda la simulación está implementada en el fichero *crear_arbol_RRT.m*. Leer el fichero y entenderlo. Organizar las ventanas de Windows de forma que se puedan ver a la vez la ventana de Matlab y la gráfica de la simulación. Ejecutar el fichero.

Cabe resaltar que el fichero contiene el comando *pause* en la línea 28, que para la ejecución del programa hasta que se apriete una tecla. De esta forma se puede apreciar mejor la expansión del árbol. Este comando es útil la primera vez que se simula. Para las ejecuciones posteriores se puede borrar o comentar.

Ir ejecutando la simulación apretando alguna tecla hasta que el robot encuentre el objetivo.

Pegar la gráfica en el documento.

Repetir el proceso cinco veces en total. Utilizar los comandos *tic* y *toc* para evaluar la duración de la búsqueda. El coste (camino recorrido) se puede obtener introduciendo "*goal.coste*".

El sesgo inicial en la práctica es de 5% hacia el objetivo (ver explicación del fichero `crear_pto_aleatorio_RRT` abajo). Probar con 0%. Ejecutar 5 veces guardar los gráficos. Registra el tiempo de simulación con *tic* y *toc*. Probar con 10%. Ejecutar 5 veces guardar los gráficos. Registra el tiempo de simulación con *tic* y *toc*.

Modificar las longitud de la rama (ver fichero `crear_nodo_RRT`) a 0.5, para un sesgo de 5%. Simular 5 veces y registrar los resultados. Modificarla a 1.5.

Analizar los resultados y escribir vuestras conclusiones.

A continuación, se van a describir y explicar brevemente algunas funciones utilizadas:

[punto] = `crear_pto_aleatorio_RRT(x_min, x_max, y_min, y_max, objetivo, sesgo)`: Esta función devuelve un punto aleatorio en el área delimitada por las coordenadas `x_min`, `x_max`, `y_min` e `y_max`, añadiéndole un sesgo en la dirección del objetivo para agilizar la búsqueda. En esta práctica se utilizó el sesgo de 5%.

[nuevo_nodo] = `crear_nodo_RRT(lista_puntos, entorno, longitud, x_min, x_max, y_min, y_max, objetivo, sesgo, numero)`: Esta función devuelve un nuevo nodo a partir de puntos aleatorios creados por la función `crear_pto_aleatorio_RRT`. Básicamente, crea un nuevo punto, verifica si no está dentro de un obstáculo, busca el nodo más próximo de árbol, une los dos nodos, busca el punto que está a la distancia dada por "`longitud`" en la línea. Si esta línea no corta ningún obstáculo, añade el nodo al árbol. De lo contrario repite el proceso de búsqueda. En un `lista_puntos` contiene una lista con las coordenadas de todos los puntos del árbol. `Entorno` es la matriz que describe el entorno en el que se efectúa la búsqueda. `Longitud` es la longitud deseada de las ramas. `x_min`, `x_max`, `y_min` e `y_max` delimitan el área en la que se buscan los puntos. "`sesgo`" es el sesgo en la dirección del objetivo. `Numero` es el número que se le asigna al nuevo nodo en el árbol.

5.2. RRT bidireccional

Este método se diferencia del anterior en que se crean dos árboles en lugar de uno. El primero parte desde el punto inicial. El segundo desde el objetivo.

En cada iteración se le añade una nueva rama a cada uno. La búsqueda se termina cuando los árboles se encuentran.

Para ejecutar el método RRT con árbol doble se debe introducir:

`crear_arbol_RRT_Bi;`

Hacer 5 búsquedas con el RRT bidireccional. Comparar los costes del camino y el tiempo de búsqueda con el RRT clásico.

5.3.RRT- Connect

Este algoritmo se implementó para encontrar un camino más corto de forma más rápida, así como para suavizar el camino obtenido por un RRT clásico. introduce unos cambios con respecto al RRT clásico:

1. Es bidireccional, al menos en la versión original.
2. Las ramas no se expanden solo una vez, sino o bien hasta tocar un obstáculo, o bien hasta alcanzar el punto aleatorio que generó la rama.
3. Cada vez que se crea un nuevo nodo se verifica si algún nodo del otro árbol es “visible”. Si esto ocurre, se unen los árboles y con esto se termina la búsqueda.

Ejecutar cinco veces:

`crear_arbol_RRT_Connect`

Comparar coste y tiempo con otros métodos.

5.4.RRT*

Este algoritmo, al igual que el anterior, se ha desarrollado para mejorar la suavidad y el coste del camino obtenidos por el RRT básico. El usuario ha de especificar otro parámetro adicional: *la vecindad*. Se trata de una distancia.

La expansión del árbol en este método se puede dividir en las siguientes fases:

1. Al generar un nuevo nodo aleatorio, al igual que en el RRT básico, se genera una rama desde el nodo más cercano.
2. Se verifica el coste del nuevo desde el inicio pasando por todos los de la vecindad. Se elige el más corto de todos.
3. Se compara el coste de cada nodo de la vecindad a través del nuevo nodo con el valor inicial. Si es menos costos a través del nuevo nodo, se modifica el árbol.

Ejecutar cinco veces:

`crear_arbol_RRT_Star`

Comparar coste y tiempo con otros métodos

Originalmente, el número de iteraciones está fijado a 500 (variable `max_iteraciones` en el archivo).

Asignarle el valor de 200 y 100.

Comparar coste y tiempo con otros métodos.